

DEEP DIVE • RETRIEVAL ARCHITECTURES

Vectorless RAG

vs Traditional RAG

When to use each, and why it matters for your AI system



Traditional

Vectorless



Krish Naik

YouTube • @krishnaik06

What We'll Cover

Five sections — by the end you'll know exactly which to pick

01

How Traditional RAG Works

Chunking, embeddings, vector similarity

02

How Vectorless RAG Works

Hierarchical trees, LLM-driven navigation

03

Side-by-Side Comparison

Latency, cost, scale, reasoning, explainability

04

When to Use Each

Decision heuristics for real production systems

05

Hybrid Architecture

Why the best systems use both together

The Core Problem RAG Solves



Why we can't just feed everything to an LLM

- **Context windows are limited**

Even 1M tokens can't hold a corporate knowledge base.

- **Long context degrades quality**

LLMs lose recall in the middle of huge prompts.

- **It's expensive**

Tokens cost money. Less context = lower bills.

THE RAG IDEA

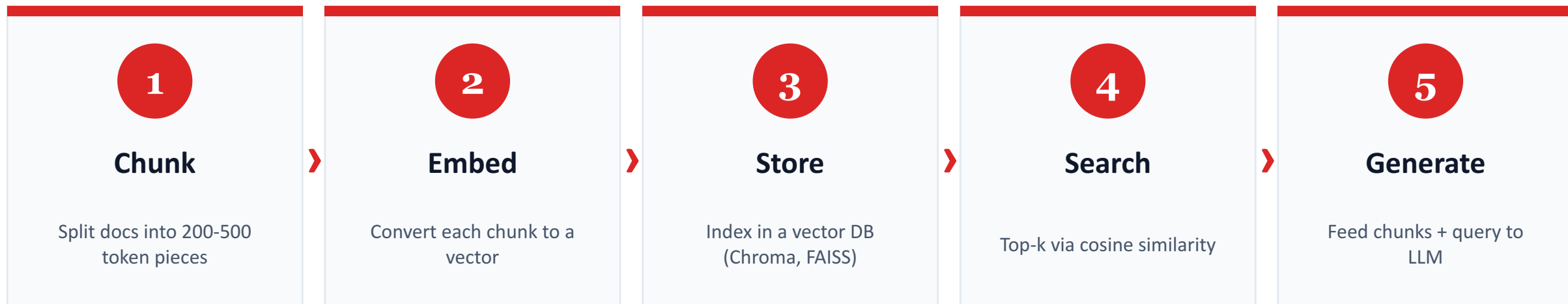
Retrieve relevant context on demand

Instead of stuffing everything into the prompt, fetch only the pieces that matter for THIS query, THEN ask the LLM.

→ *The big question: HOW do we retrieve?*

Traditional RAG: The Pipeline

Document → vector → similarity search → LLM



EXAMPLE

Query: "What is HDFC Bank's net interest margin?" → Embed query → find 3 most similar chunks → LLM answers from those chunks.

Traditional RAG: The Real Picture

It's powerful — but it has known failure modes



STRENGTHS

Massive scale

Millions of docs, millisecond lookups

Mature ecosystem

Chroma, FAISS, Pinecone, Qdrant, Weaviate

Cheap retrieval

One embedding + ANN search per query

Great for factoids

Short, lookup-style questions

Domain agnostic

Works on any text — blogs, tickets, PDFs



WEAKNESSES

Chunking destroys context

"As defined in Section 3.2..." — meaningless when retrieved alone

Similarity ≠ relevance

Embeddings can match wrong things confidently

No cross-section reasoning

Can't answer "compare risks vs mitigations"

Hard to explain

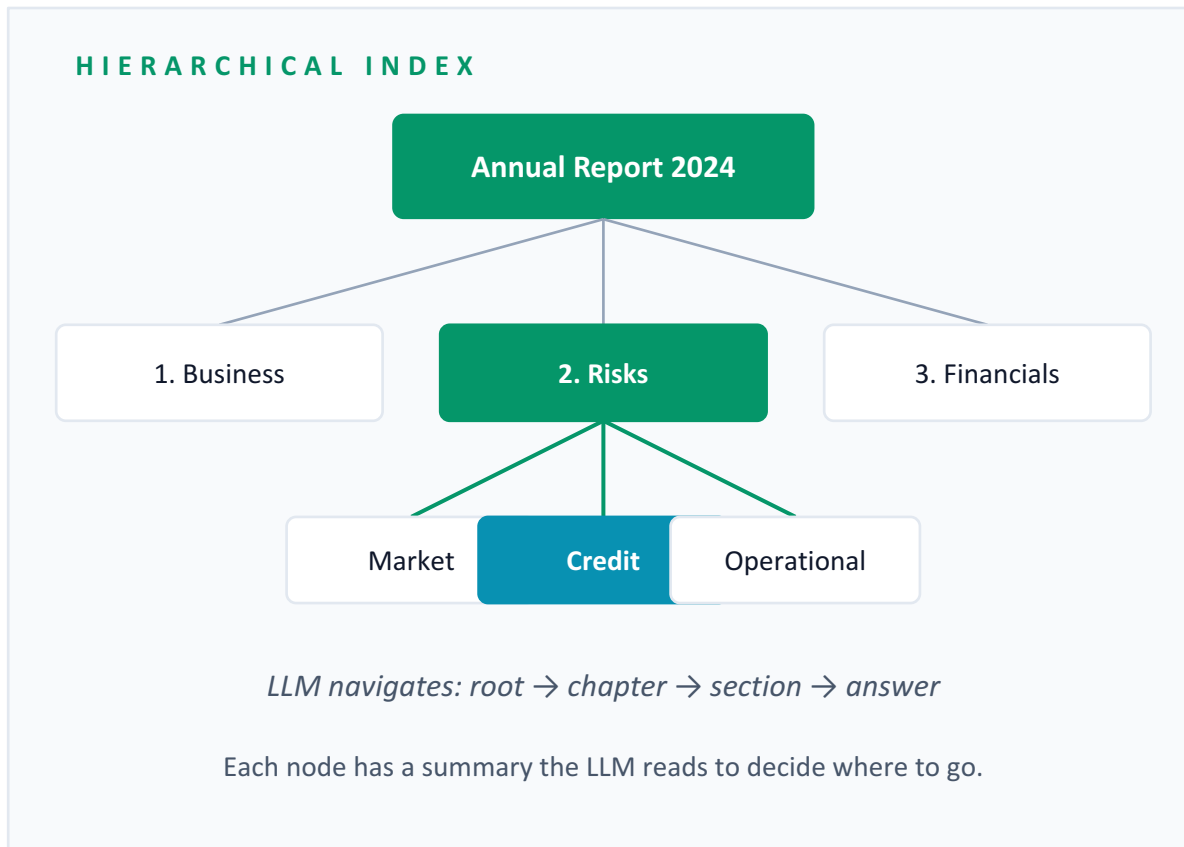
Why was this chunk picked? Cosine score isn't an answer

Embedding drift

Model changes = re-embed everything

Vectorless RAG: Reasoning Through Structure

Skip embeddings entirely — let the LLM navigate the document like a human would



How the LLM navigates

- 1 Build the tree**
Parse doc structure (headings, sections); generate summaries at each node — done once, offline
- 2 LLM reads root summary**
“Which chapter is most likely to contain the answer?”
- 3 Descend the tree**
Repeat at each level until reaching a leaf section
- 4 Read full section**
No chunking — full context preserved
- 5 Answer + cite path**
Returns answer AND the navigation path it took

Vectorless RAG: The Real Picture

Different tradeoffs — better for some workloads, worse for others



STRENGTHS

Preserves document context

Sections stay whole — no broken references

Cross-section reasoning

LLM can compare, contrast, synthesize

Explainable retrieval

Returns the navigation path, not a cosine score

No embedding pipeline

Skip the embed/index/refresh complexity

Plays well with structure

Reports, contracts, filings, textbooks shine



WEAKNESSES

Higher per-query cost

Multiple LLM calls to traverse the tree

Higher latency

Several hundred ms to a few seconds per query

Doesn't scale to millions

Works for tens to thousands of docs, not internet-scale

Needs structured docs

Random blog posts? Tree adds little value

Less mature tooling

PageIndex and a few others — ecosystem is young

Side-by-Side: The Honest Comparison

Eight dimensions that matter in production systems

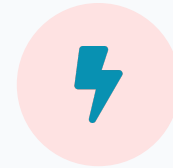
Dimension	Traditional RAG	Vectorless RAG
Scale	Millions of docs ✓	Tens to thousands
Latency / query	Milliseconds ✓	Hundreds of ms — seconds
Cost / query	Cheap ✓ (1 embed lookup)	Higher (multiple LLM calls)
Cross-section reasoning	Weak	Strong ✓
Explainability	Cosine score (opaque)	Navigation path ✓
Best for	Factoid Q&A, mixed corpora	Long structured docs
Setup complexity	Embedding pipeline + DB	Tree builder, no DB
Ecosystem maturity	Very mature ✓	Emerging

Use Traditional RAG when...



Massive heterogeneous corpora

Millions of mixed-format docs — blogs, tickets, transcripts, knowledge base articles.



Latency-critical apps

Chatbots, search-as-you-type, voice assistants. Every millisecond counts.



Short factoid queries

“What's the warranty period?” “Who's the CEO?” The answer lives in one chunk.



Cost-sensitive at scale

Thousands of queries per minute. Embedding lookups are pennies; LLM-tree-walks aren't.

Use Vectorless RAG when...



Long, structured documents

Annual reports, 10-Ks, legal contracts, regulatory filings, research papers, textbooks.



Reasoning > similarity

“Compare risk factors in section 7 to mitigations in section 12.” Pure similarity can't do this.



Explainability is required

Compliance, audit, legal, financial advisory. Show your work — not just the answer.



Chunking destroys meaning

Cross-references like “as defined in §3.2” lose their meaning when retrieved in isolation.

The 2-Question Decision Heuristic

Ask these two questions; the answer picks itself

Q1

Is the answer in one localized chunk, or does it require navigating structure?

LOCALIZED

STRUCTURAL

Q2 • How many documents?

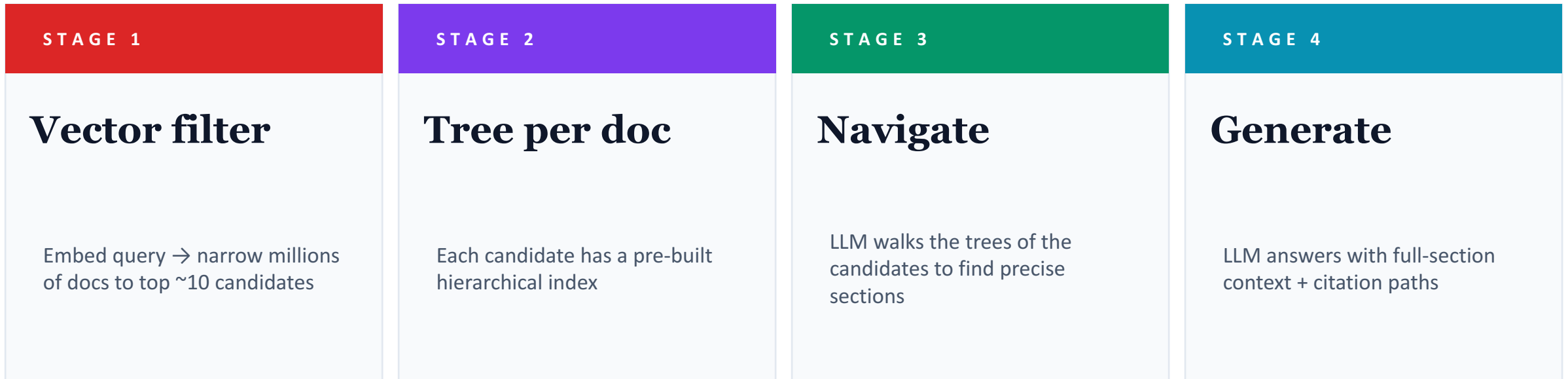
- **Millions:** Traditional RAG
Vector search wins on scale
- **Hundreds–thousands:** Either works
Pick based on query type
- **Tens of long docs:** Likely vectorless better

Q2 • Are docs well-structured?

- **Yes (TOC, headings):** Vectorless RAG
Tree navigation is reliable
- **Mixed structure:** Hybrid
Vector to filter, tree to reason
- **Unstructured:** Traditional RAG

The Hybrid Pattern

Most serious production systems combine both — here's how



Why this wins: scale from vectors, precision from trees, explainability from navigation paths.

Key Takeaways

Five things to remember from this video

1

Traditional RAG = scale + speed

Vector similarity is unbeatable for millions of docs and millisecond lookups.

2

Vectorless RAG = reasoning + structure

LLM-driven tree navigation preserves context and explains itself.

3

They're not competitors

They're complementary. Pure vector search and pure tree navigation are both extremes.

4

The right pick depends on the doc, not the hype

Long structured filings → vectorless. Mixed knowledge base → vector. Big system → hybrid.

5

Production systems are going hybrid

Vectors narrow the search space; trees do the precise reasoning inside it.

THANK YOU FOR WATCHING

Build smarter retrieval

Pick the right tool for the document, not the trend.



Subscribe

@krishnaik06



Code on GitHub

krishnaik06



Learn more

krishnaik.in